Assignment #5 BST Binary Search Tree – Implementation and Performance Analysis

Part 2: Insertion-Order Experiments

For part 2, I compared how ascending, random, and balanced insertion affected the height and search performance of a Binary Search Tree (BST). From my results, ascending insertion produced the tallest trees because the keys were inserted from smallest to large. Since each new key was greater than the previous one, the noeds were always added to the right, creating a straight chain instead of branching into left and right subtrees. For example, N = 2,000, the ascending BST had a height of 1,999, compared to an average height of 23.8 for random insertion and a height of 10 for balanced insertion.

As the number of nodes increased, I noticed that the height of the ascending BST increased much faster than the other two strategies. When N increased from 100 to 2,000, the ascending tree height increased from 99 to 1,999, while the balanced tree only increased from 6 to 10. This explains why ascending insertion can result in O(N) search time in the worst case, while balanced insertion allows for O(logN) search time. Random insertion also had much slower height growth than ascending insertion and generally has O(logN) expected search time. However, random insertion does not guarantee a balanced tree because some branches may still become longer than others. For example, when N = 1,000, the random BST had an average height of approximately 21 across 10 trials, while the balanced BST had a height of 9.

The average successful search comparisons followed a similar pattern to the tree heights. Ascending insertion required the most comparisons, followed by random insertion, while balanced insertion required the fewest. When N = 2,000, the ascending BST required an average of 1,000.5 comparisons, compared to 13.27 for random insertion and 9.98 for balanced insertion. This happens because taller trees generally have longer paths to reach a node, requiring more comparisons during a search. However, I also learned that tree height does not completely determine average search performance because it only measures the longest path. The average depth of all nodes gives a better understanding how many comparisons are needed to find a key.

Part 3: Node-Depth Distribution

When I compared the node-depth distributions for the 1,000-node BSTs, I noticed that each insertion strategy produced different results. Ascending insertion had one node at every depth from 0 to 999 because the tree formed a straight chain. Random insertion had most nodes around the middle depths, with a maximum depth of 20. Balanced insertion had most nodes around depth 8 and 9, with a maximum depth of 9. This shows that balanced insertion kept the nodes much closer to the root compared to the other strategies.

I also learned that the two BSTs can have similar heights but different average search performance. This is because height only measures the deepest node, not where all the other nodes are located. A tree with more nodes closer to the root will generally require fewer comparisons when searching for keys. This means average node depth is a better way to understand overall search performance when every key is searched equally often.

Average node depth and successful search comparisons are also directly related. Since searching starts at the root, which has a depth of 0 but requires one comparison, the number of comparisons is always the node depth plus 1. This can be written as C _avg = *D _avg* + 1 when each key is searched exactly once. For example, in my experiment with 100 nodes, the balanced BST had an average depth of 4.8 and an average of 5.8 comparisons, which confirms this relationship.

Overall, my results from Parts 2 and 3 showed that the order of inserting keys affects the shape of a BST and how efficiently it performs searches. Ascending insertion had the worst performance because it created a long chain with a greater height and node depths. Random insertion performed much better, but it did not always create a balanced tree. Balanced insertion gave the best results because it kept the tree shorter and the nodes closer to the root, which required fewer comparisons when searching. From these experiments, I learned that keeping a BST balanced becomes more important as the number of nodes increases.

AI Interaction Record

I used ChatGPT throughout this assignment to help me understand my Binary Search Tree code and complete the experiments. I originally asked ChatGPT how to do the insertion order experiments, but I was not logged into an account at the time, so the part of the conversation was not saved in the chat history I submitted. Afterward, I used ChatGPT to review my Python code, help create the graphs using Matplotlib, and figure out how to organize and write my analysis for Parts 2 and 3.

One example of something I verified was when ChatGPT explained that average successful search comparisons should equal node depth plus 1. I checked this with my experimental results and confirmed that my balanced BST with 100 nodes had an average depth of 4.8 and an average of 5.8 comparisons. Overall, AI helped me better understand the assignment, check my work, and organize my analysis.